OZON tech

Из чего состоит компилятор Razor и как его готовить

Георгий Круглов
Старший разработчик информационных систем

xaml/axaml

```
xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

<TextBox Text="{Binding FirstName}" />
<TextBox Text="{Binding FirstName, Mode=TwoWay}” />
```

razor

```
@using MudBlazor
@using System.Collections.Generic
@using Phazor.Components

<TextBox Text="@FirstName"/>
<TextBox @bind-Text="@FirstName"/>
```

```
@inherits RazorStringBuilder

update table
set updated_at = now()
@if (string.IsNullOrEmpty(Data) is false)
{
    @:, data = :data
}
where id = :id

@code {
    public required string? Data { get; set; }
}
```

Про что поговорим

- Что такое Razor и как устроен его компилятор

Про что поговорим

- Что такое Razor и как устроен его компилятор
- Как использовать этот компилятор для своих нужд

Про что поговорим

- Что такое Razor и как устроен его компилятор
- Как использовать этот компилятор для своих нужд
- Что интересного можно с ним сделать

01

Что такое Razor

Razor – язык разметки

- Совмещает XML-like синтаксис для разметки и C# для шаблонизации
- Позволяет встраивать C# код как “тело класса”

Razor – язык разметки

- Совмещает XML-like синтаксис для разметки и C# для шаблонизации
- Позволяет встраивать C# код как “тело класса”
- В основном используется в контексте WebUI на C# (Blazor, Razor Pages, ASP.NET MVC)

Razor – язык разметки

- Совмещает XML-like синтаксис для разметки и C# для шаблонизации
- Позволяет встраивать C# код как “тело класса”
- В основном используется в контексте WebUI на C# (Blazor, Razor Pages, ASP.NET MVC)
- Имеет open-source компилятор
    - Реализован как source generator
    - Преобразует документы Razor кода в Razor компоненты (C# классы)
    - Имеет модульную архитектуру

Razor – язык разметки

Hello.razor

```
<div style="display: flex; justify-content: center; align-items: center">
   <p>
      Hello, @Name
   </p>
</div>

@code
{
   [Parameter]
   public string? Name { get; set; }
}
```

Razor – язык разметки

Hello_razor.g.cs

```
public class Hello : ComponentBase
{
    protected override void BuildRenderTree(RenderTreeBuilder builder)
    {
        builder.OpenElement(0, "div");
        builder.AddAttribute(1, "style", "display: flex; justify-content: center; align-items: center");
        builder.OpenElement(2, "p");
        builder.AddMarkupContent(3, "\n        Hello, ");
        builder.AddContent(4, Name);
        builder.CloseElement();
        builder.CloseElement();
    }

    [Parameter]
    public string? Name { get; set; }
}
```

Почему Razor?

Почему Razor?

- Нативность в экосистеме .NET

Почему Razor?

- Нативность в экосистеме .NET
- Мы хотим не шаблонизировать C# код, а **шаблонизировать с помощью** C# кода

Почему Razor?

- Нативность в экосистеме .NET
- Мы хотим не шаблонизировать C# код, а **шаблонизировать с помощью** C# кода
- Компилятор Razor – уже реализован, и его исходный код находится в открытом доступе
- Компилятор Razor реализован на основе Roslyn и source generator’ов

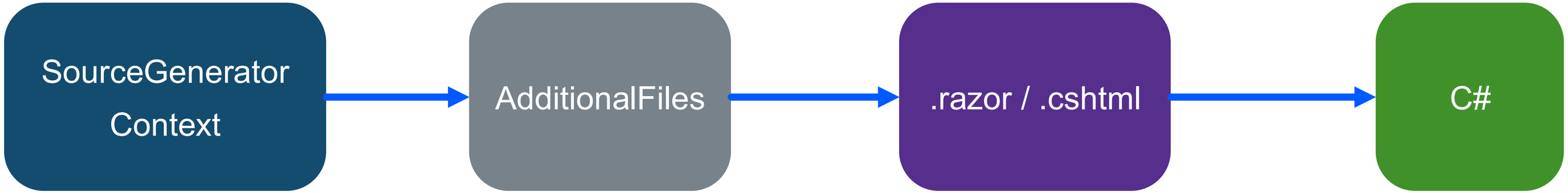
Почему Razor?

- Нативность в экосистеме .NET
- Мы хотим не шаблонизировать C# код, а **шаблонизировать с помощью** C# кода
- Компилятор Razor – уже реализован, и его исходный код в находится открытом доступе
- Компилятор Razor реализован на основе Roslyn и source generator’ов
- Принцип Razor – генерировать C# классы, которые что-либо шаблонизируют

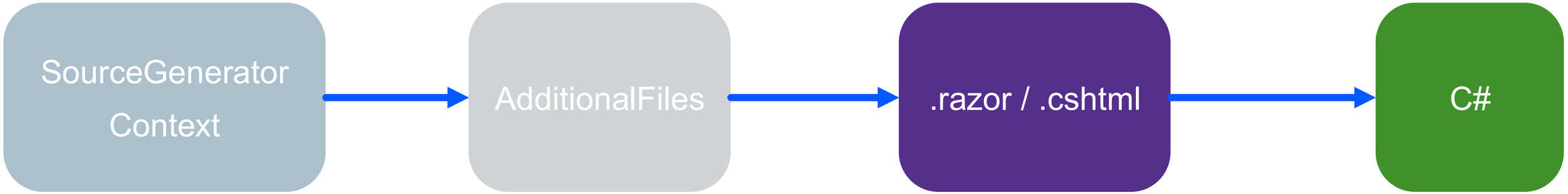
02

Устройство компилятора Razor

Устройство компилятора Razor



Устройство компилятора Razor



Устройство компилятора Razor

- RazorEngine
    - Фасад, выполняющий различные преобразования Razor кода

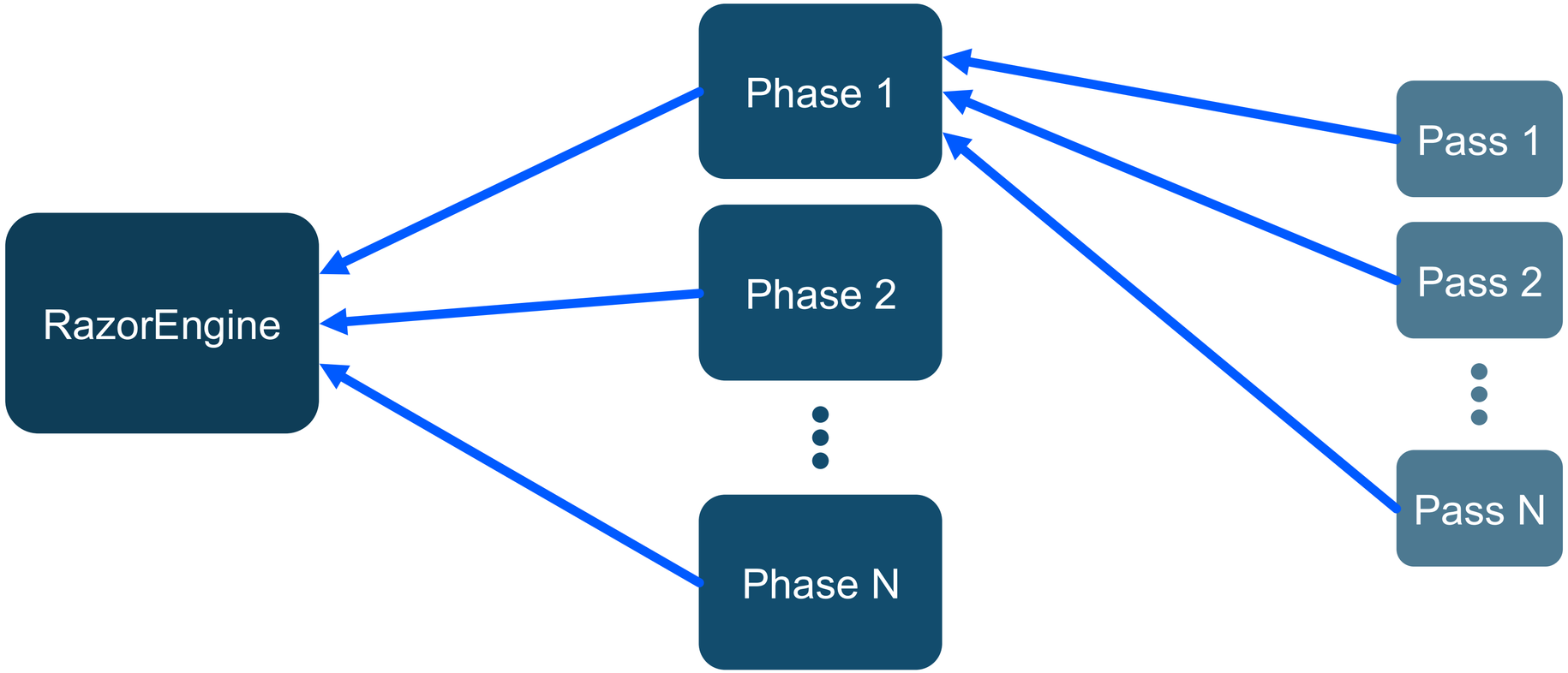
Устройство компилятора Razor

- RazorEngine
  - Фасад, выполняющий различные преобразования Razor кода

- RazorProjectEngine
  - Верхнеуровневый фасад
  - Настраивает RazorEngine
  - Настраивает контекст, в котором выполняется RazorEngine

RazorEngine

Устройство компилятора Razor



Проходы (Passes)

Устройство компилятора Razor

- Проходы RazorEngine (Passes) – фичи движка/языка (IRazorEngineFeature)
- Добавляя/удаляя различные фичи, мы можем кастомизировать поведение компилятора
- Получаются из движка при инициализации фаз (Phases)

```
Passes = Engine
    .GetFeatures<IRazorDocumentClassifierPass>()
    .OrderByAsArray(p => p.Order);
```

Фазы (Phases)

Устройство компилятора Razor

- Шаги преобразования Razor документа в C# код
- Выполняются над RazorCodeDocument – контекстом компиляции Razor документа
- RazorCodeDocument хранит конфигурации и различные представления Razor кода

```
public interface IRazorEnginePhase
{
    RazorEngine Engine { get; }

    void Initialize(RazorEngine engine);

    void Execute(
        RazorCodeDocument codeDocument,
        CancellationToken cancellationToken);
}
```

Представления Razor кода

Устройство компилятора Razor

- RazorSourceDocument – строковое представление

Представления Razor кода

Устройство компилятора Razor

- RazorSourceDocument – строковое представление
- RazorSyntaxTree – синтаксическое дерево

Представления Razor кода

Устройство компилятора Razor

- RazorSourceDocument – строковое представление
- RazorSyntaxTree – синтаксическое дерево
- IntermediateSyntaxNode – промежуточное синтаксическое дерево. Фазы и проходы преобразуют его, добавляя различные типизированные синтаксические фичи.

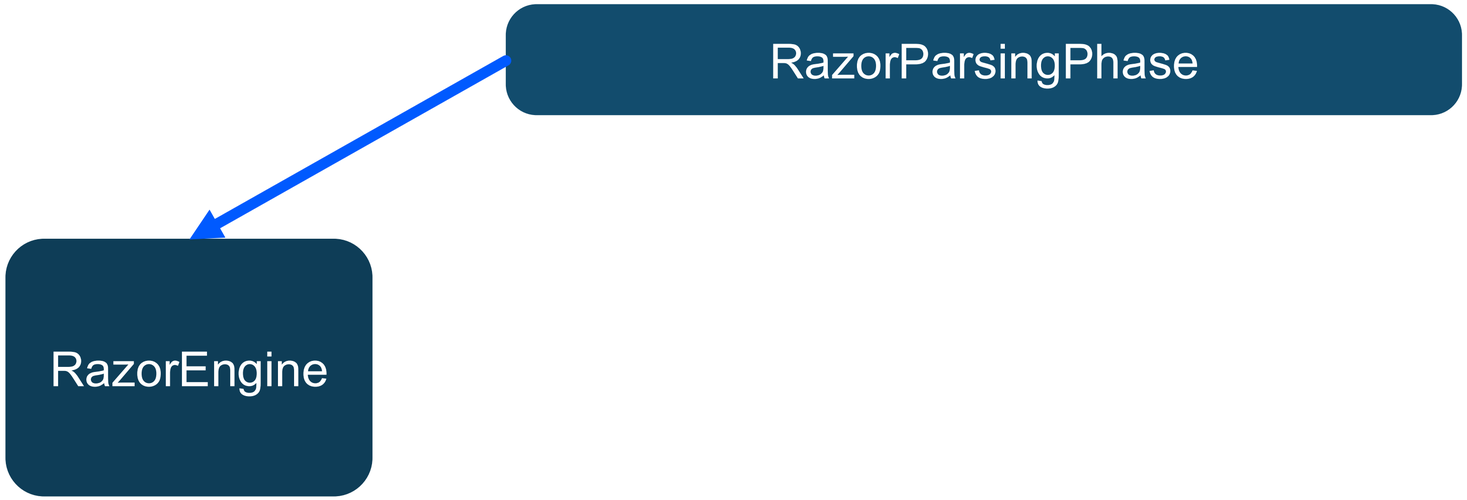
Фазы компиляции Razor

Устройство компилятора Razor

RazorEngine

RazorParsingPhase

Устройство компилятора Razor

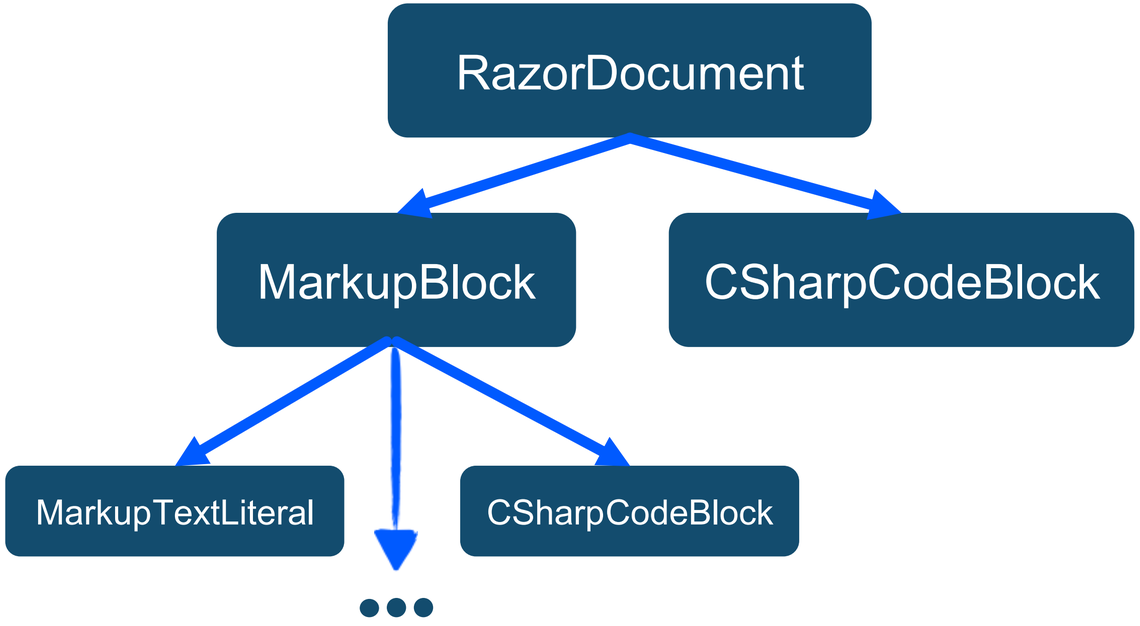


RazorParsingPhase

Устройство компилятора Razor

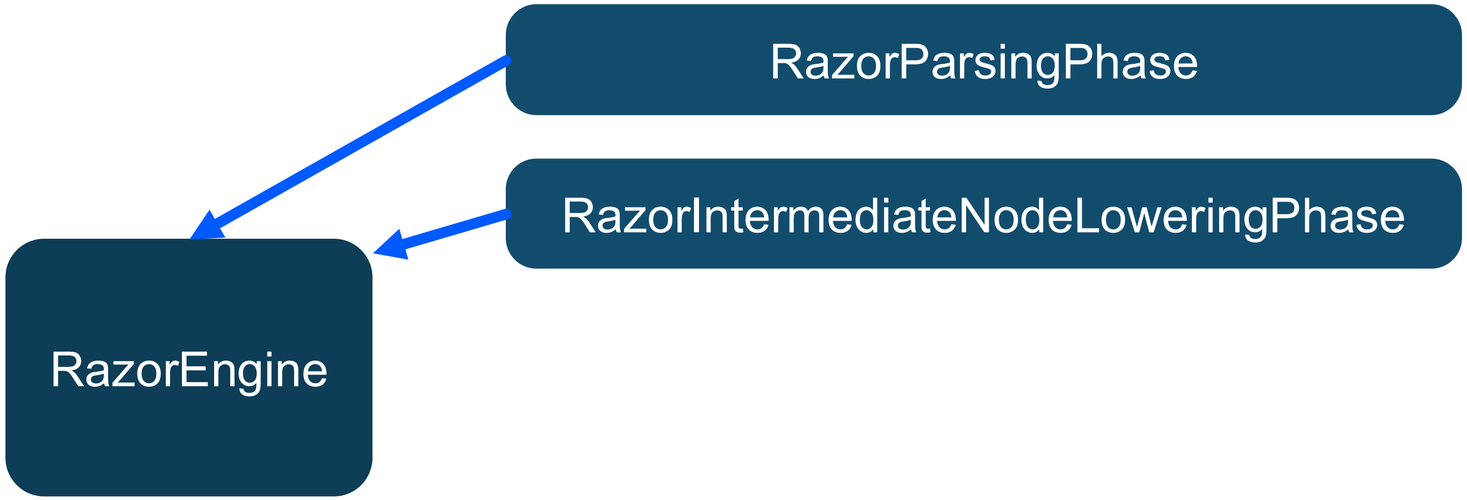
- Преобразует RazorSourceDocument в RazorSyntaxTree
- Добавляет его в RazorCodeDocument

```
<div style="display: flex;
            justify-content: center;
            align-items: center">
   <p>
      Hello, @Name
   </p>
</div>

@code
{
   [Parameter]
   public string? Name { get; set; }
}
```



RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor



RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

- Превращает RazorSyntaxTree в IntermediateNode

RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

- Превращает RazorSyntaxTree в IntermediateNode
- Собирает import’ы, преобразуя их в using директивы

_Imports.razor

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Web.Virtualization
@using Microsoft.AspNetCore.Components.WebAssembly.Http
@using Microsoft.JSInterop
@using MudBlazor
@using Phazor.Components
@using Itmo.Dev.Asap.Frontend
@using Itmo.Dev.Asap.Frontend.Presentation.Markup.Shared
@using Itmo.Dev.Asap.Frontend.Presentation.Components
@using Itmo.Dev.Asap.Frontend.Presentation.Components.Authorization
@using Itmo.Dev.Asap.Frontend.Presentation.Components.Grid
@using Itmo.Dev.Asap.Frontend.Presentation.Components.Pickers
@using Microsoft.AspNetCore.Authorization
```

RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

- Превращает RazorSyntaxTree в IntermediateNode
- Собирает import’ы, преобразуя их в using директивы
- Разбивает Razor код на элементы
  - Директивы (не типизированные)
  - HTML контент

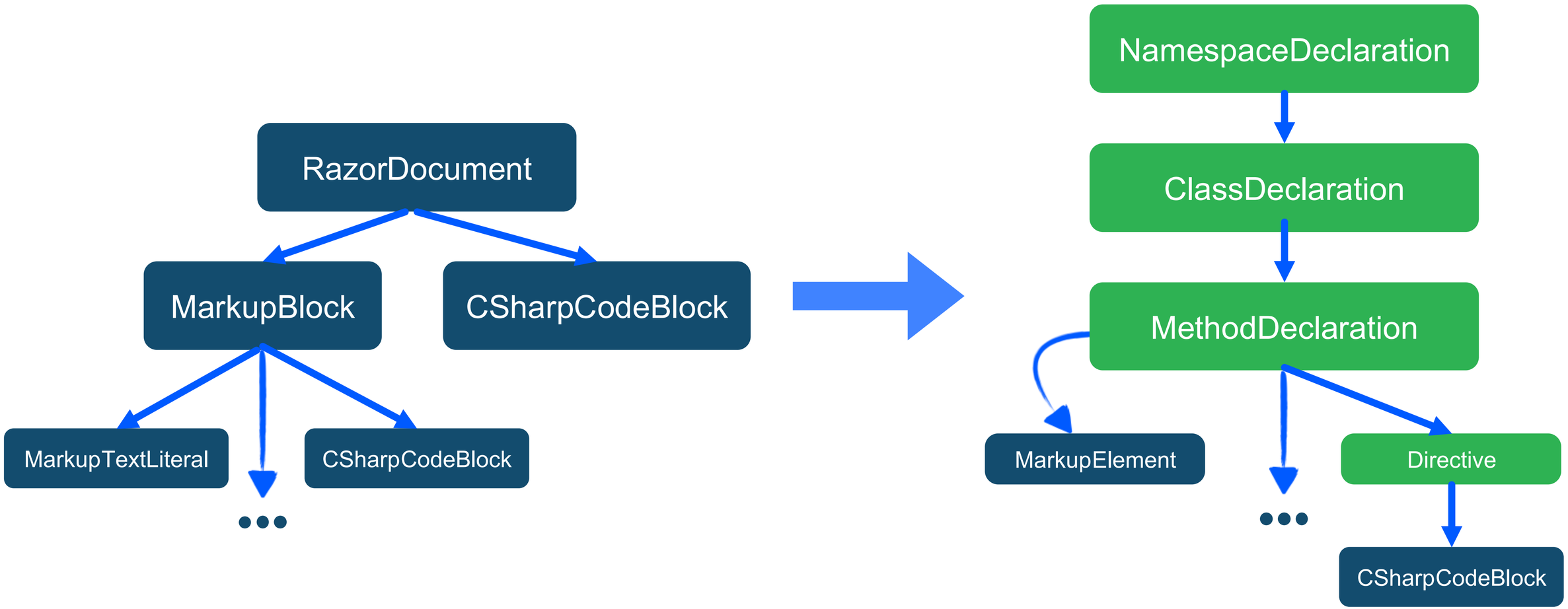
RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

- Превращает RazorSyntaxTree в IntermediateNode
- Собирает import’ы, преобразуя их в using директивы
- Разбивает Razor код на элементы
  - Директивы (не типизированные)
  - HTML контент
- Получаем структуру более похожую на целевой формат Razor документа

RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor



RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

```
<div style="display: flex; justify-content: center; align-items: center">
   <p>
      Hello, @Name
   </p>
</div>

@code
{
   [Parameter]
   public string? Name { get; set; }
}
```

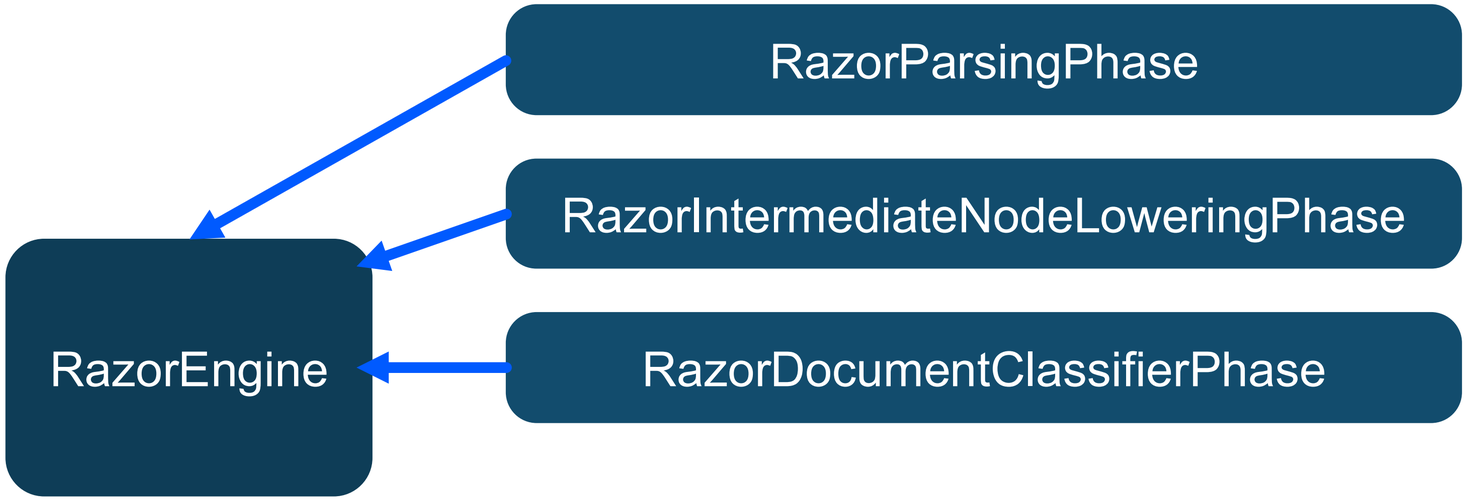
RazorIntermediateNodeLoweringPhase

Устройство компилятора Razor

```
public class Hello : ComponentBase
{
   protected override void BuildRenderTree(RenderTreeBuilder builder)
   {
      builder.OpenElement(0, "div");
      builder.AddAttribute(1, "style", "display: flex; justify-content: center; align-items: center");
      builder.OpenElement(2, "p");
      builder.AddMarkupContent(3, "\n       Hello, ");
      builder.AddContent(4, Name);
      builder.CloseElement();
      builder.CloseElement();

         [Parameter]
          public string? Name { get; set; }
   }
}
```

RazorDocumentClassifierPhase

Устройство компилятора Razor



RazorDocumentClassifierPhase

Устройство компилятора Razor

- Фактически определяется целевой формат Razor кода
- Для кастомизации нужно определить собственный проход
- Появляется возможность кастомизации структуры документа

```
public class CustomDocumentClassifierPass(AnalyzerConfigOptions options)
    : DocumentClassifierPassBase
```

Фильтрация Razor документов

RazorDocumentClassifierPhase

```
protected override bool IsMatch(
    RazorCodeDocument codeDocument,
    DocumentIntermediateNode documentNode)
{
    return codeDocument.FileKind.IsComponent();
}
```

Создание CodeTarget

RazorDocumentClassifierPhase

```
protected override CodeTarget CreateTarget(
    RazorCodeDocument codeDocument,
    RazorCodeGenerationOptions options)
{
    return new CustomCodeTarget(options, TargetExtensions);
}
```

Создание CodeTarget

RazorDocumentClassifierPhase

```
public class CustomCodeTarget : DefaultCodeTarget
{
    public override IntermediateNodeWriter CreateNodeWriter()
    {
        return new CustomNodeWriter();
    }
}
```

Реализация OnDocumentStructureCreated

RazorDocumentClassifierPhase

```
protected override void OnDocumentStructureCreated(
    RazorCodeDocument codeDocument,
    NamespaceDeclarationIntermediateNode ns,
    ClassDeclarationIntermediateNode cls,
    MethodDeclarationIntermediateNode method)
```

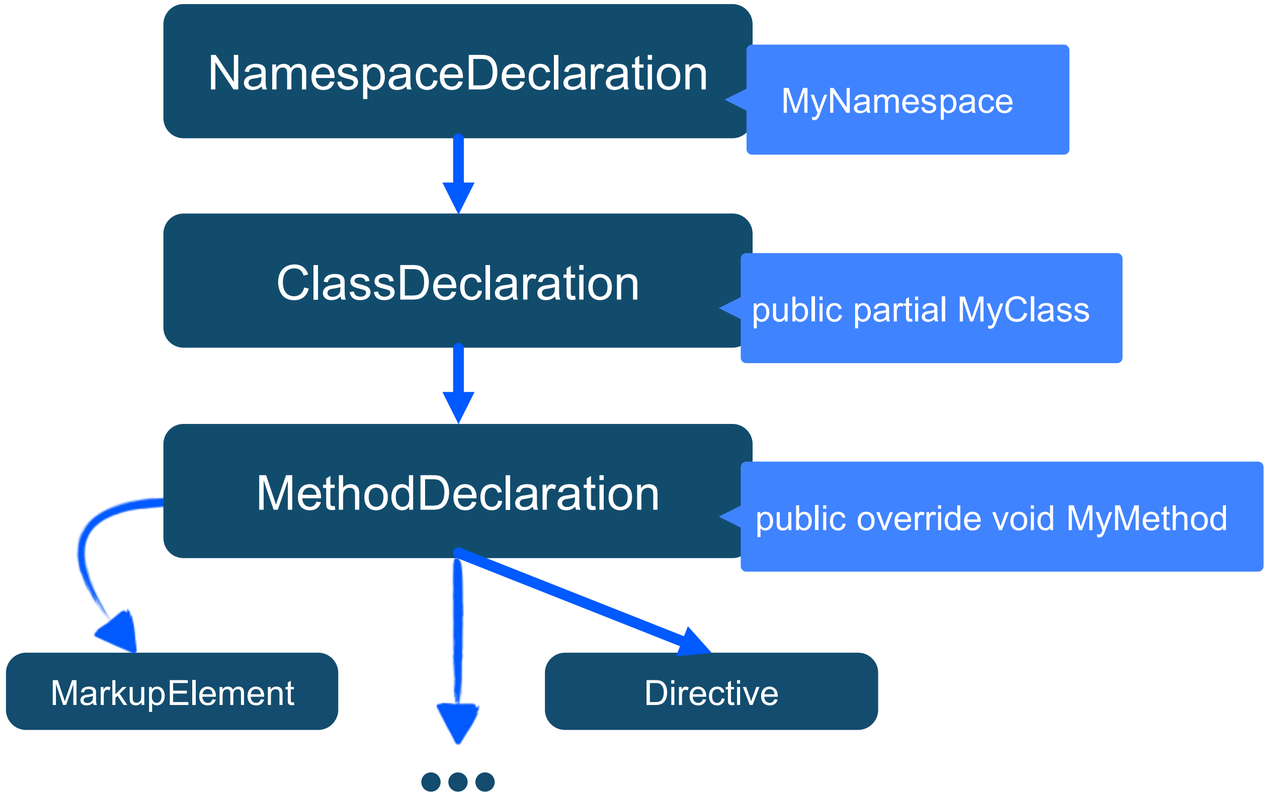
Реализация OnDocumentStructureCreated

RazorDocumentClassifierPhase

```
protected override void OnDocumentStructureCreated(
    RazorCodeDocument codeDocument,
    NamespaceDeclarationIntermediateNode ns,
    ClassDeclarationIntermediateNode cls,
    MethodDeclarationIntermediateNode method)
{
        ns.Name = projectItem.HintNamespace;

        cls.Modifiers = cls.Modifiers.Add("public").Add("partial");
        cls.Name = projectItem.HintClassName;

        method.Modifiers = method.Modifiers.Add("public").Add("override");
        method.ReturnType = "void";
        method.Name = "Build";

        var methodParameter = new MethodParameter("builder", "global::System.Text.StringBuilder");
        method.Parameters = method.Parameters.Add(methodParameter);
}
```

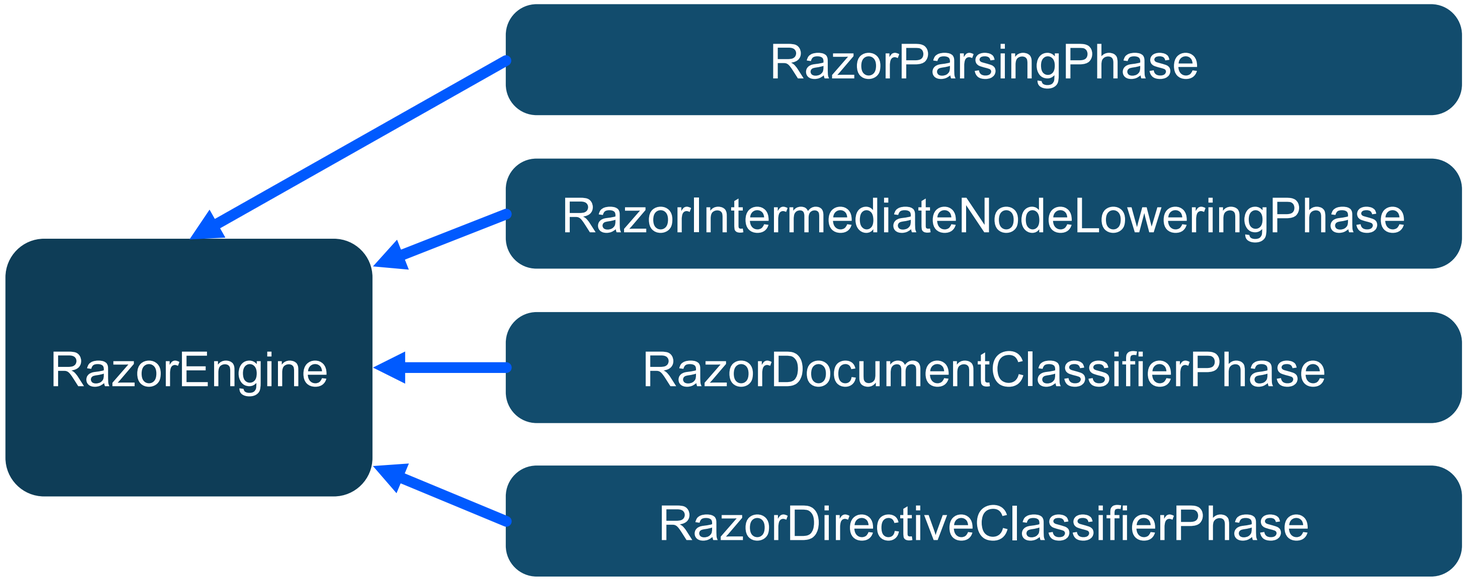
Реализация OnDocumentStructureCreated

RazorDocumentClassifierPhase



RazorDirectiveClassifierPhase

Устройство компилятора Razor



RazorDirectiveClassifierPhase

Устройство компилятора Razor

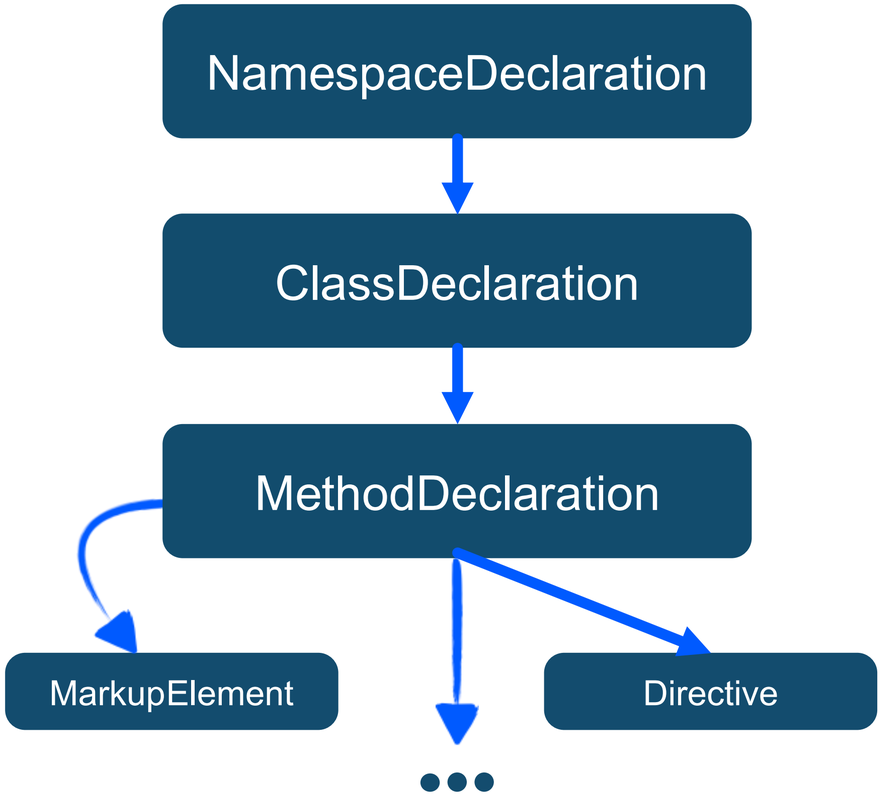
- Фаза парсинга типизированных директив
  - @inject
  - @code
  - @implements

- Директивы регистрируются как фичи движка

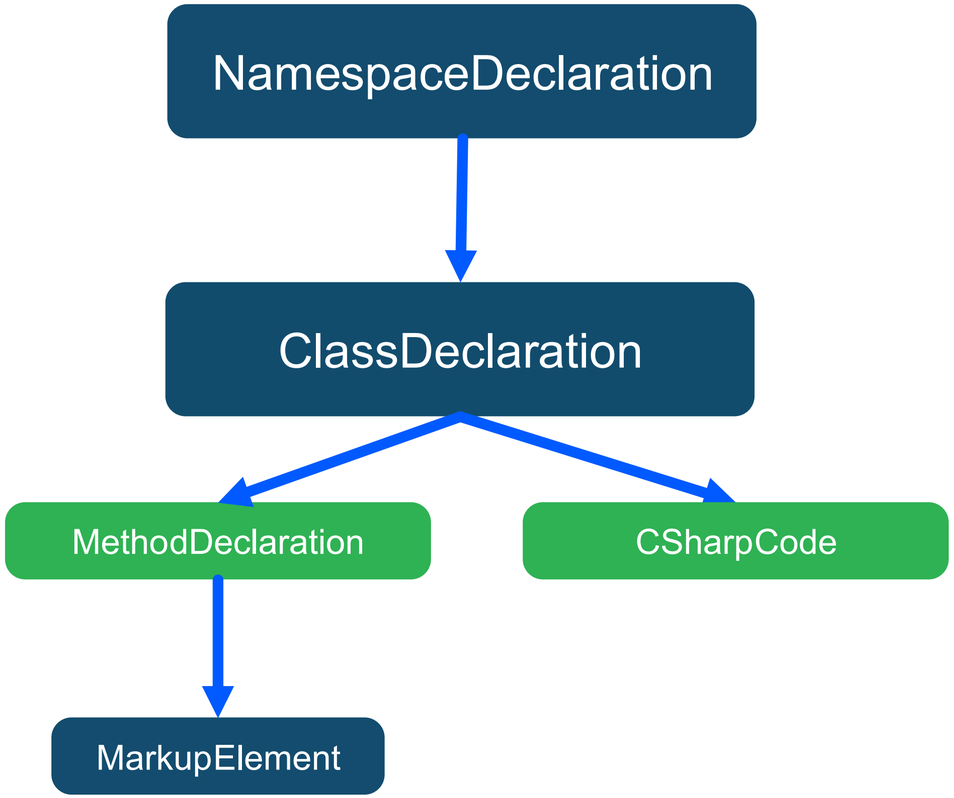
```
FunctionsDirective.Register(builder);
ComponentCodeDirective.Register(builder);
```

RazorDirectiveClassifierPhase

Устройство компилятора Razor

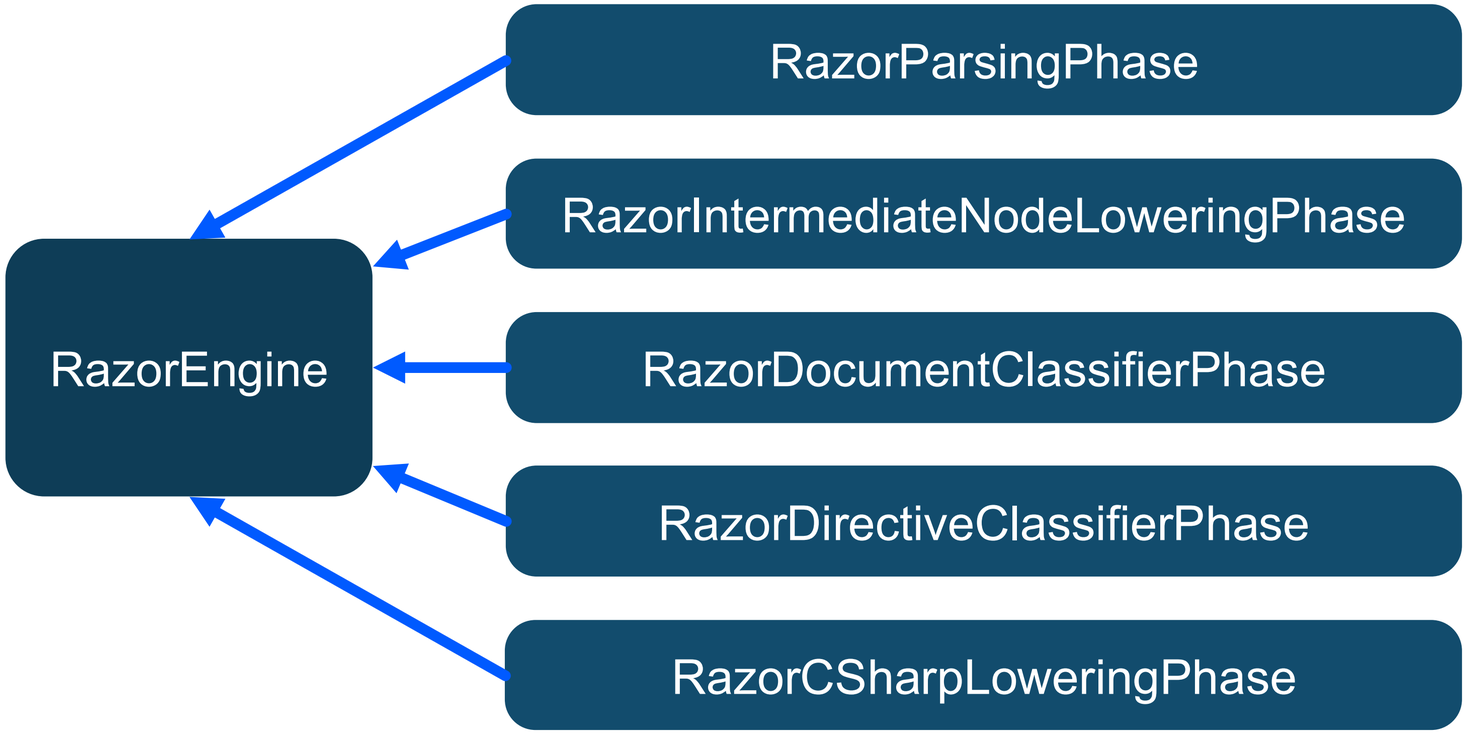



```
FunctionsDirective.Register(builder);
ComponentCodeDirective.Register(builder);
```



RazorCSharpLoweringPhase

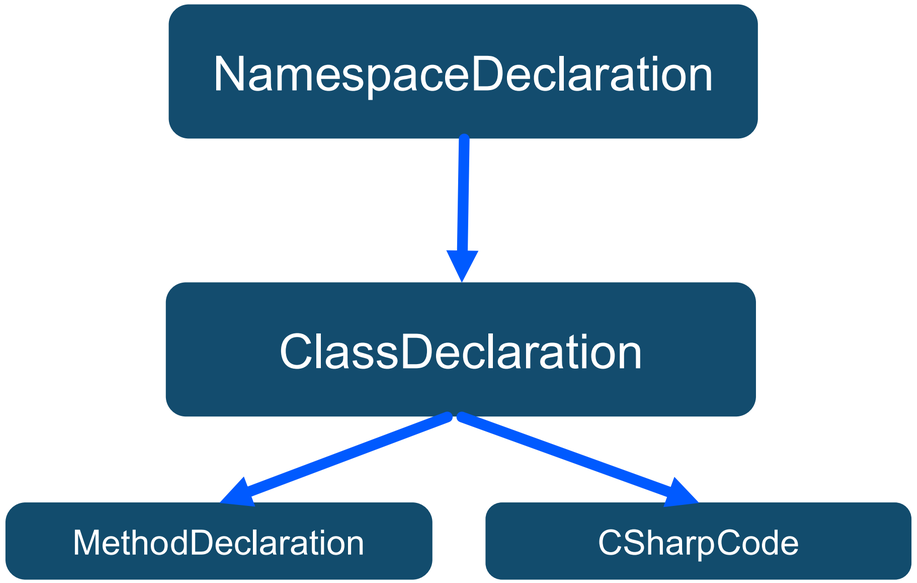
Устройство компилятора Razor



RazorCSharpLoweringPhase

Устройство компилятора Razor

- Использует ранее созданный CodeTarget, чтобы создать IntermediateNodeWriter
- Использует IntermediateNodeWriter, чтобы преобразовать IntermediateNode в текст




```
namespace MyNamespace;

public partial class MyClass
{
    public partial override void MyMethod()
    {
        ...
    }
}
```

IntermediateNodeWriter

RazorCSharpLoweringPhase

- Визитор по IntermediateNode
- Имеет методы посещения всех видов промежуточных узлов, принимающие CodeRenderingContext
  - Содержит CodeWriter – фактически StringBuilder
  - Позволяет задать source mapping

```
public class CustomNodeWriter : IntermediateNodeWriter
{
   public override void WriteUsingDirective(CodeRenderingContext context, UsingDirectiveIntermediateNode node)
      => context.CodeWriter.WriteUsing(node.Content);

   public override void WriteCSharpExpression(CodeRenderingContext context, CSharpExpressionIntermediateNode node)
   {
      using (context.BuildLinePragma(node.Source, suppressLineDefaultAndHidden: true))
      {
         ...
      }
   }
}
```

IntermediateNodeWriter

RazorCSharpLoweringPhase

```
using (context.BuildLinePragma(node.Source, suppressLineDefaultAndHidden: true))
{
    …
}
```

```
#line 2 “/.../UpdateQuery.razor”
        builder.Append("\nupdate table\nset updated_at = now()\n");
```

03

RazorStringBuilder

Зачем?

RazorStringBuilder

- Шаблонизация строк со сложной логикой
  - Более наглядная, чем через обычный C# код и StringBuilder
- SQL запросы

Логика

RazorStringBuilder

```
@inherits RazorStringBuilder

update table
set updated_at = now()
@if (string.IsNullOrEmpty(Data) is false)
{
    @:, data = :data
}
where id = :id

@code {
    public required string? Data { get; set; }
}
```

```
public partial class TableUpdate : RazorStringBuilder
{
    public override void Build(StringBuilder builder)
    {
        builder.Append("update table\nset updated_at = now()\n");
        if (string.IsNullOrEmpty(Data) is false)
        {
            builder.Append("    ");
            builder.Append(", data = :data\n");
        }

        builder.Append("where id = :id\n\n");
    }

    public required string? Data { get; set; }
}
```

Реализация NodeWriter

RazorStringBuilder

```
public class RazorStringBuilderNodeWriter : IntermediateNodeWriter
{
    ...
}
```

Реализация NodeWriter

RazorStringBuilder

```
private static void WriteCode(CodeRenderingContext context, IntermediateNode node)
{
    using (context.BuildLinePragma(node.Source, suppressLineDefaultAndHidden: true))
    {
        foreach (IntermediateNode child in node.Children)
        {
            if (child is CSharpIntermediateToken token)
            {
                context.CodeWriter.Write(token.Content);
            }
            else
            {
                context.RenderNode(child);
            }
        }

        context.CodeWriter.WriteLine();
    }
}
```

```
@if (string.IsNullOrEmpty(Data) is false)
{
    ...
}
```

Реализация NodeWriter

RazorStringBuilder

```
private static void WriteContent(CodeRenderingContext context, IntermediateNode node)
{
    using (context.BuildLinePragma(node.Source, suppressLineDefaultAndHidden: true))
    {
        context.CodeWriter.Write("builder.Append(\"");

        foreach (IntermediateNode child in node.Children)
        {
            if (child is HtmlIntermediateToken token)
            {
                context.CodeWriter.Write(token.Content.Replace("\n", "\\n"));
            }
        }

        context.CodeWriter.WriteLine("\");");
    }
}
```

update table
set updated_at = now()

Реализация NodeWriter

RazorStringBuilder

```
private static void WriteValue(CodeRenderingContext context, IntermediateNode node)
{
    using (context.BuildLinePragma(node.Source, suppressLineDefaultAndHidden: true))
    {
        context.CodeWriter.Write("builder.Append(");

        foreach (IntermediateNode child in node.Children)
        {
            if (child is CSharpIntermediateToken token)
            {
                context.CodeWriter.Write(token.Content);
            }
            else
            {
                context.RenderNode(child);
            }
        }

        context.CodeWriter.WriteLine(");");
    }
}
```

```
@:, data = '@Data'
```

Реализация NodeWriter

RazorStringBuilder

```
public class RazorStringBuilderNodeWriter : IntermediateNodeWriter
{
   public override void WriteUsingDirective(CodeRenderingContext context, UsingDirectiveIntermediateNode node)
      => context.CodeWriter.WriteUsing(node.Content);

   public override void WriteCSharpExpression(CodeRenderingContext context, CSharpExpressionIntermediateNode node)
      => WriteValue(context, node.Children);

   public override void WriteCSharpCode(CodeRenderingContext context, CSharpCodeIntermediateNode node)
      => WriteCode(context, node.Children);

   public override void WriteHtmlContent(CodeRenderingContext context, HtmlContentIntermediateNode node)
      => WriteContent(context, node.Children);

   public override void WriteHtmlAttribute(CodeRenderingContext context, HtmlAttributeIntermediateNode node)
      => WriteContent(context, node.Children);

   public override void WriteHtmlAttributeValue(CodeRenderingContext context, HtmlAttributeValueIntermediateNode node)
      => WriteContent(context, node.Children);

   ...
```

Результат

RazorStringBuilder

```
@inherits RazorStringBuilder

update table
set updated_at = now()
@if (string.IsNullOrEmpty(Data) is false)
{
    @:, data = '@Data'
}
where id = :id

@code {
    public required string? Data { get; set; }
}
```

```
public partial class TableUpdate : RazorStringBuilder
{
    public override void Build(StringBuilder builder)
    {
        builder.Append("update table\nset updated_at = now()\n");
        if (string.IsNullOrEmpty(Data) is false)
        {
            builder.Append("    ");
            builder.Append(", data = '");
            builder.Append(Data);
            builder.Append("'\n");
        }

        builder.Append("where id = :id\n\n");
    }

    public required string? Data { get; set; }
}
```

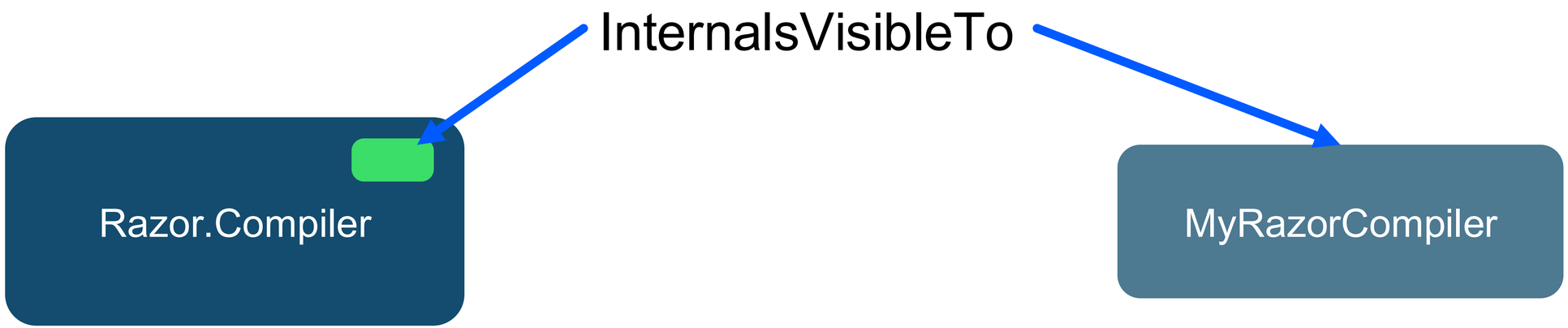
04

Как мне использовать компилятор Razor в своём проекте?

Наименее инвазивный способ

Как мне использовать компилятор Razor в своём проекте?

- Сборка Razor.Compiler является .NET.Sdk.Razor
- Можно пропатчить её через Cecil



Наименее инвазивный способ

Как мне использовать компилятор Razor в своём проекте?

Плюсы

- Не нужно модифицировать код компилятора
- Не нужно распространять дополнительные сборки

Наименее инвазивный способ

Как мне использовать компилятор Razor в своём проекте?

Плюсы

- Не нужно модифицировать код компилятора
- Не нужно распространять дополнительные сборки

Минусы

- Модификаторы доступа проверяются и в рантайме
- Придётся патчить сборку Razor.Compiler на всех клиентских устройствах

Наименее инвазивный способ

Как мне использовать компилятор Razor в своём проекте?

Плюсы

- Не нужно модифицировать код компилятора
- Не нужно распространять дополнительные сборки

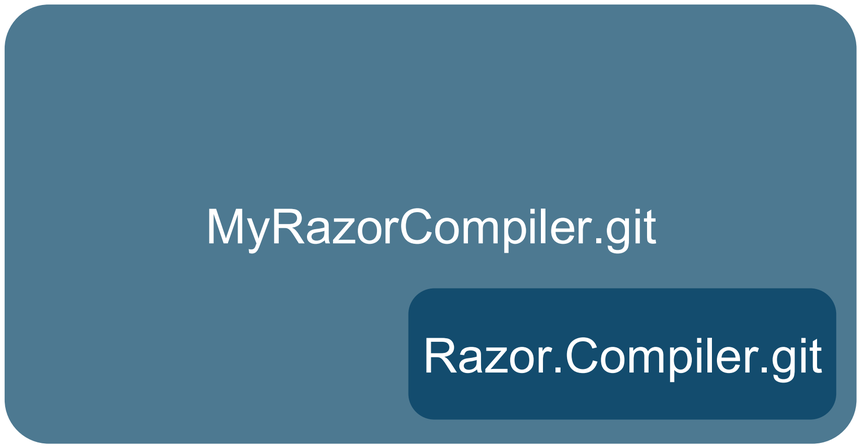
Минусы

- Модификаторы доступа проверяются и в рантайме
- Придётся патчить сборку Razor.Compiler на всех клиентских устройствах
- Код нашего компилятора завязывается на конкретную версию Razor.Compiler

Сабмодули

Как мне использовать компилятор Razor в своём проекте?

- Можно подключить репозиторий dotnet/razor в качестве сабмодуля в свой проект



Сабмодули

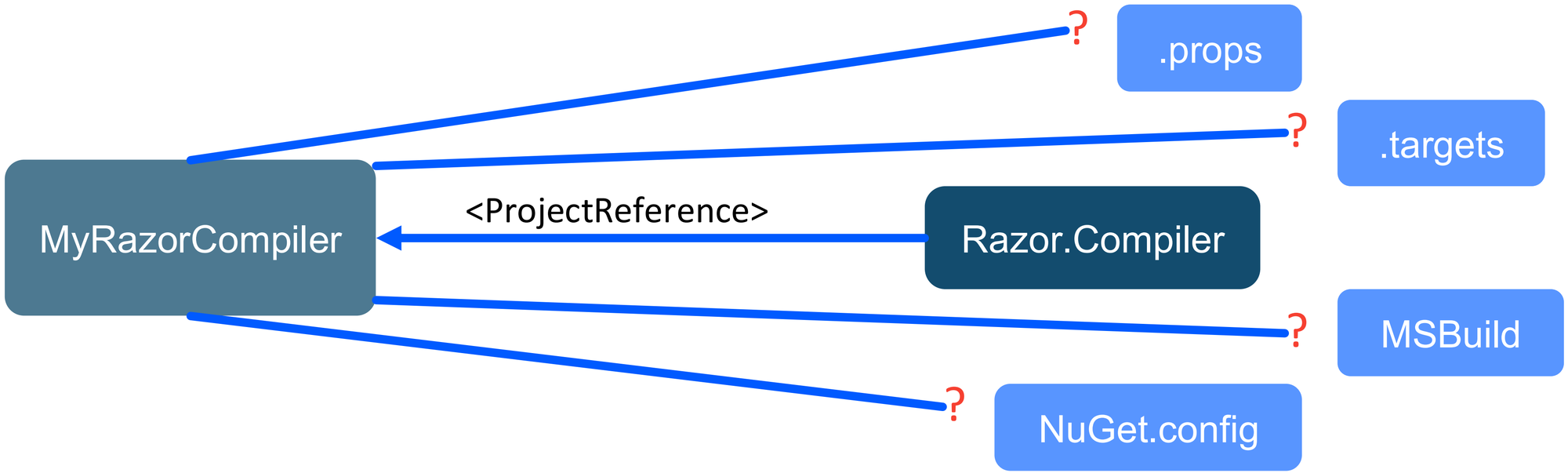
Как мне использовать компилятор Razor в своём проекте?

Плюсы

- Не нужно как-либо патчить SDK
- Версия Razor.Compiler у клиентов всегда совпадает с используемой нашим компилятором

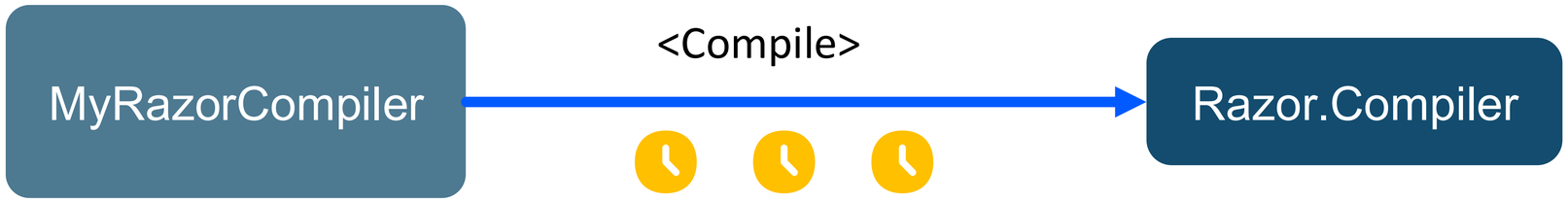
Сабмодули

Как мне использовать компилятор Razor в своём проекте?



Сабмодули

Как мне использовать компилятор Razor в своём проекте?



Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Microsoft.CodeAnalysis.Razor.Compiler

ronimizy.Razor.Sdk.Compiler

Модифицируем Versions.props

Процедурная генерация SDK

- В репозитории компилятора используются версии пакетов, доступные только в репозиториях Microsoft
- Используем SDK MSBuild

```
<PropertyGroup>
  <MicrosoftCodeAnalysisCommonPackageVersion>5.0.0-1.25209.6</...>
  <MicrosoftCodeAnalysisCSharpPackageVersion>5.0.0-1.25209.6</...>
</PropertyGroup>
```

↓

```
<PropertyGroup>
  <RonimizyCodeAnalysisVerion>4.13.0</RonimizyCodeAnalysisVerion>
  <MicrosoftCodeAnalysisCommonPackageVersion>$(RonimizyCodeAnalysisVerion)</...>
  <MicrosoftCodeAnalysisCSharpPackageVersion>$(RonimizyCodeAnalysisVerion)</...>
</PropertyGroup>
```

Модифицируем Versions.props

Процедурная генерация SDK

```
public class PropsReplacement
{
    public PropsReplacement(string fileName)
    {
        FileName = fileName;
    }

    public string FileName { get; }

    public Dictionary<string, string> Properties { get; init; } = new();
}
```

```
private static readonly PropsReplacement VersionsPropsReplacement = new("eng/Versions.props")
{
    Properties = new()
    {
        [CodeAnalysisVersionProp] = "4.13.0",
        ["MicrosoftCodeAnalysisCSharpPackageVersion"] = CodeAnalysisVersionPropRef,
        ["MicrosoftCodeAnalysisCommonPackageVersion"] = CodeAnalysisVersionPropRef,
    }
};
```

Модифицируем Versions.props

Процедурная генерация SDK

```
private async Task ApplyPropsReplacement(Workspace workspace, PropsReplacement replacement)
{
   string filePath = Path.Combine(
      Path.GetDirectoryName(workspace.CurrentSolution.FilePath!)!,
      replacement.FileName);

   var root = ProjectRootElement.Open(filePath)!;

   foreach (KeyValuePair<string, string> replacementProperty in replacement.Properties)
   {
      root.AddProperty(replacementProperty.Key, replacementProperty.Value);
   }

   root.Save();
}
```

Удаляем лишние генераторы

Процедурная генерация SDK

```
SyntaxNode[] generatorClasses = root
    .DescendantNodes()
    .OfType<ClassDeclarationSyntax>()
    .Where(node => IsGeneratorClass(node, semanticModel))
    .Cast<SyntaxNode>()
    .ToArray();

if (generatorClasses is [])
    continue;

root = root.RemoveNodes(generatorClasses, SyntaxRemoveOptions.KeepNoTrivia)!;
```

```
private static bool IsGeneratorClass(ClassDeclarationSyntax node, SemanticModel semanticModel)
{
    if (semanticModel.GetDeclaredSymbol(node) is not INamedTypeSymbol symbol)
        return false;

    return symbol.GetAttributes().Any(attribute => attribute.AttributeClass?.Name == "GeneratorAttribute");
}
```

Меняем название сборок у проектов

Процедурная генерация SDK

- Мы не можем загрузить на nuget.org пакет с префиксом Microsoft

```
private async Task ModifyPrefixesAsync(Workspace workspace, Project project, string targetPrefix)
{
   var projectRoot = ProjectRootElement.Open(project.FilePath!)!;

   if (TryGetAssemblyNameWithChangedPrefix(project.AssemblyName, targetPrefix, out string assemblyName))
   {
      ProjectPropertyGroupElement propertyGroup = projectRoot.AddPropertyGroup();
      propertyGroup.AddProperty("AssemblyName", assemblyName);
      propertyGroup.AddProperty("PackageId", assemblyName);
   }

      ...
}
```

Microsoft.CodeAnalysis.Razor.Compiler → ronimizy.Razor.Sdk.Compiler

Скрываем зависимости на другие проекты

Процедурная генерация SDK

- Нам будет проще сложить сборки проектов-зависимостей напрямую в итоговый NuGet пакет

```
<ItemGroup>
  <ProjectReference Include="...Microsoft.AspNetCore.Razor.Utilities.Shared.csproj" />
</ItemGroup>
```

↓

```
<ItemGroup>
  <ProjectReference
    Include="...Microsoft.AspNetCore.Razor.Utilities.Shared.csproj"
    PrivateAssets="all"/>
</ItemGroup>
```

Скрываем зависимости на другие проекты

Процедурная генерация SDK

```
private async Task InlineDependenciesAsync(Workspace workspace, Project project, string? targetPrefix)
{
   var projectRoot = ProjectRootElement.Open(project.FilePath!)!;

   IEnumerable<ProjectItemElement> projectReferences = projectRoot.ItemGroups
      .SelectMany(x => x.Items)
      .Where(x => x.ElementName == "ProjectReference");

   foreach (ProjectItemElement reference in projectReferences)
   {
      reference.AddMetadata("PrivateAssets", "All");
   }

   await AddProjectReferenceDllsAsync(workspace, projectRoot.AddItemGroup(), project, targetPrefix);

...
```

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- Чтобы сборки проектов-зависимостей содержались в результирующем пакете, нужно добавить их вручную

```
private async Task InlineDependenciesAsync(Workspace workspace, Project project, string? targetPrefix)
{
   var projectRoot = ProjectRootElement.Open(project.FilePath!)!;

   IEnumerable<ProjectItemElement> projectReferences = projectRoot.ItemGroups
      .SelectMany(x => x.Items)
      .Where(x => x.ElementName == "ProjectReference");

   foreach (ProjectItemElement reference in projectReferences)
   {
      reference.AddMetadata("PrivateAssets", "All");
   }

   await AddProjectReferenceDllsAsync(workspace, projectRoot.AddItemGroup(), project, targetPrefix);

...
```

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- При подключении сборок используем названия с изменённым префиксом

```
private async Task AddProjectReferenceDllsAsync(
    Workspace workspace,
    ProjectItemGroupElement itemGroup,
    Project project,
    string? targetPrefix)
{
    foreach (ProjectReference projectReference in project.ProjectReferences)
    {
        Project referencedProject = workspace.CurrentSolution.GetProject(projectReference.ProjectId)!;

        TryGetAssemblyNameWithChangedPrefix(
            referencedProject.AssemblyName,
            targetPrefix,
            out string assemblyName);

        ...
```

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- Объявляем свойство содержащее путь до сборки проекта-зависимости

```
...

var itemName = $"A{assemblyName.Replace('.', '_')}Path";
itemGroup.AddItem(itemName, @$”$(OutputPath)**\{assemblyName}.dll");

...
```

My.Assembly.Name → <AMy_Assembly_NamePath>
./bin/Release/MyAssemblyName.dll
</AMy_Assembly_NamePath>

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- Добавляем сборку проекта-зависимости в результирующий пакет

```
...

ProjectItemElement item = itemGroup.AddItem("None", include: $"@({itemName})");
item.AddMetadata("Pack", "true");
item.AddMetadata("PackagePath", $@"lib\%(RecursiveDir){assemblyName}.dll");
item.AddMetadata("Visible", “false");

...
```

↓

```
<None Include="@(AMy_Assembly_NamePath)">
  <Pack>true</Pack>
  <PackagePath>...</PackagePath>
  <Visible>false</Visible>
</None>
```

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- Выполняем рекурсивно для всех транзитивных проектов

```
    ...

        await AddProjectReferenceDllsAsync(workspace, itemGroup, referencedProject, targetPrefix);
    }
}
```

Включаем сборки проектов-зависимостей

Процедурная генерация SDK

- Также нам нужно подключить все пакеты из проектов-зависимостей

```
        ...

        await AddProjectReferenceDllsAsync(workspace, projectRoot.AddItemGroup(), project, targetPrefix);

    ProjectItemGroupElement packageReferenceGroup = projectRoot.AddItemGroup();

    foreach (ProjectReference reference in project.ProjectReferences)
    {
        Project referencedProject = workspace.CurrentSolution.GetProject(reference.ProjectId)!;
        await AddProjectReferencePackageReferences(workspace, packageReferenceGroup, referencedProject);
    }

    projectRoot.Save();
}
```

Открываем все типы/методы

Процедурная генерация SDK

```
public class AccessModifierSyntaxRewriter : CSharpSyntaxRewriter
{
   public override SyntaxNode? VisitClassDeclaration(ClassDeclarationSyntax node)
   {
      if (ShouldIgnore(node) is false && TryOpenModifiers(
            node.Modifiers,
            out SyntaxTokenList resultModifiers,
            replaceProtected: true,
            replaceEmpty: true))
      {
         node = node.WithModifiers(resultModifiers);
      }
      return base.VisitClassDeclaration(node);
   }

   ...
```

Открываем все типы/методы

Процедурная генерация SDK

```
private async Task OpenInternalsAsync(Workspace workspace, Project project)
{
   foreach (Document document in project.Documents)
   {
      if (document.SupportsSyntaxTree is false)
         continue;

       ...
```

Открываем все типы/методы

Процедурная генерация SDK

```
private async Task OpenInternalsAsync(Workspace workspace, Project project)
{
   foreach (Document document in project.Documents)
   {
      if (document.SupportsSyntaxTree is false)
         continue;

               SyntaxTree syntaxTree = (await document.GetSyntaxTreeAsync())!;

         SyntaxNode root = await syntaxTree.GetRootAsync();
         SemanticModel semanticModel = (await document.GetSemanticModelAsync())!;
         root = new AccessModifierSyntaxRewriter(ExcludedFromOpenNamespaces, semanticModel).Visit(root);

         Document newDocument = document.WithSyntaxRoot(root);
         string formattedCode = root.NormalizeWhitespace(eol: Environment.NewLine).ToFullString();

         await File.WriteAllTextAsync(newDocument.FilePath!, formattedCode, Encoding.UTF8);
      }

      ...
```

Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Плюсы:

- Не тянем сложный build пайплайн в кастомные компиляторы

Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Плюсы:

- Не тянем сложный build пайплайн в кастомные компиляторы
- Не собираем лишний код при сборке кастомного компилятора

Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Плюсы:

- Не тянем сложный build пайплайн в кастомные компиляторы
- Не собираем лишний код при сборке кастомного компилятора
- Доступны все типы/методы из SDK

Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Плюсы:

- Не тянем сложный build пайплайн в кастомные компиляторы
- Не собираем лишний код при сборке кастомного компилятора
- Доступны все типы/методы из SDK
- Простой процесс перехода на новую версию компилятора

Процедурная генерация SDK

Как мне использовать компилятор Razor в своём проекте?

Плюсы:

- Не тянем сложный build пайплайн в кастомные компиляторы
- Не собираем лишний код при сборке кастомного компилятора
- Доступны все типы/методы из SDK
- Простой процесс перехода на новую версию компилятора

Минусы:

- Сам процесс модификации и сборки изначального SDK – не тривиальный

Но, я собрал его в NuGet пакет

05

Как мне сделать собственный компилятор Razor?

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

```
[Generator]
public class MyRazorGenerator : IIncrementalGenerator
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

1. Достаём из сборки только .razor файлы

```
public void Initialize(IncrementalGeneratorInitializationContext context)
{
    var analyzerConfigOptions = context.AnalyzerConfigOptionsProvider;
    var additionalTexts = context.AdditionalTextsProvider;

    var razorFiles = additionalTexts
        .Where(static text => text.Path.EndsWith(".razor", StringComparison.OrdinalIgnoreCase));

    ...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

2. Вычисляем SourceGeneratorProjectItem’ы

```
...

var projectItems = razorFiles
    .Combine(context.AnalyzerConfigOptionsProvider)
    .Select((tuple, ct) => ComputeProjectItems(tuple.Left, tuple.Right, ct))
    .ReportDiagnostics(context);

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

2. Вычисляем SourceGeneratorProjectItem’ы

```
new SourceGeneratorProjectItem(
    basePath: "/",
    filePath: additionalText.Path,
    relativePhysicalPath: additionalText.Path,
    fileKind: FileKinds.GetFileKindFromPath(additionalText.Path),
    additionalText: additionalText,
    cssScope: null);
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

3. Достаём импорты и компоненты

```
...

var importFiles = projectItems
    .Where(static item => item.FileKind.IsComponentImport());

var componentProjectItems = projectItems
    .Where(static item => item.FileKind.IsComponent())
    .Combine(importFiles.Collect())
    .Combine(analyzerConfigOptions);

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

4. Создаём RazorProjectEngine

```
...

var generatedDeclarationText = componentProjectItems
    .Select(static (tuple, cancellationToken) =>
    {
        var ((razorFile, imports), options) = tuple;

        var fileSystem = new VirtualRazorProjectFileSystem();
        fileSystem.Add(razorFile);

        foreach (var import in imports)
        {
            fileSystem.Add(import);
        }

        var builder = new RazorProjectEngineBuilder(RazorConfiguration.Default, fileSystem);

        ...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

4.1. Добавляем фазы компиляции

```
...

builder.Phases.Add(new DefaultRazorParsingPhase());
builder.Phases.Add(new DefaultRazorSyntaxTreePhase());
builder.Phases.Add(new DefaultRazorIntermediateNodeLoweringPhase());
builder.Phases.Add(new DefaultRazorDocumentClassifierPhase());
builder.Phases.Add(new DefaultRazorDirectiveClassifierPhase());
builder.Phases.Add(new DefaultRazorCSharpLoweringPhase());

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

4.2. Добавляем фичи/проходы

```
...

// SyntaxTreePhase
builder.Features.Add(new DefaultDirectiveSyntaxTreePass());
builder.Features.Add(new HtmlNodeOptimizationPass());

// IntermediateLoweringPass
builder.Features.Add(new EliminateMethodBodyPass());

// DocumentClassifierPhase
builder.Features.Add(new CustomDocumentClassifierPass(options.GlobalOptions));

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

4.2. Добавляем фичи/проходы

```
...

// SyntaxTreePhase
builder.Features.Add(new DefaultDirectiveSyntaxTreePass());
builder.Features.Add(new HtmlNodeOptimizationPass());

// IntermediateLoweringPass
builder.Features.Add(new EliminateMethodBodyPass());

// DocumentClassifierPhase
builder.Features.Add(new CustomDocumentClassifierPass(options.GlobalOptions));

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

4.3.Добавляем типизированные директивы

```
...

FunctionsDirective.Register(builder);
ImplementsDirective.Register(builder);
InheritsDirective.Register(builder);
NamespaceDirective.Register(builder);
AttributeDirective.Register(builder);
ComponentCodeDirective.Register(builder);
ComponentInjectDirective.Register(builder);

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

5. Собираем и запускаем RazorProjectEngine

```
    ...

      RazorProjectEngine projectEngine = builder.Build();

    RazorCodeDocument codeDocument = projectEngine.Process(razorFile);

    return (razorFile, codeDocument.GetCSharpDocument());
});

...
```

Реализация source generator’a

Как мне сделать собственный компилятор Razor?

6. Добавляем результат генерации в сборку

```
...

context.RegisterSourceOutput(
    generatedDeclarationText,
    (ctx, item) =>
    {
        var (projectItem, codeDocument) = item;

        SyntaxTree syntaxTree = CSharpSyntaxTree.ParseText(codeDocument.GetCSharpDocument().Text);
        SyntaxNode syntaxNode = syntaxTree.GetRoot();

        syntaxNode = syntaxNode.NormalizeWhitespace(eol: "\n");

        var text = syntaxNode.ToFullString();
        var fileName = $"{projectItem.HintNamespace}.{projectItem.HintClassName}.Razor.g.cs";

        ctx.AddSource(fileName, text);
    });
```

Подключение .razor файлов

Как мне сделать собственный компилятор Razor?

```
<Project>

  <ItemGroup Label="Setup .razor files" Condition="'$(IsRazorSdkComponent)' != 'true'">

    <RazorSdkSources Include="**/*.razor" Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)"/>

    <None Remove="@(RazorSdkSources)"/>
    <AdditionalFiles Include="@(RazorSdkSources)"/>

    <AdditionalFiles
        Update="@(RazorSdkSources)"
        HintNamespace="$([MSBuild]::ValueOrDefault('$(RootNamespace).%(RelativeDir)', '').Replace('\', '.').Replace('/', '.').Trim('.'))"/>

  </ItemGroup>


      ...



</Project>
```

Подключение .razor файлов

Как мне сделать собственный компилятор Razor?

```
<Project>

  <ItemGroup Label="Setup .razor files" Condition="'$(IsRazorSdkComponent)' != 'true'">

    <RazorSdkSources Include="**/*.razor" Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)"/>

    <None Remove="@(RazorSdkSources)"/>
    <AdditionalFiles Include="@(RazorSdkSources)"/>

    <AdditionalFiles
        Update="@(RazorSdkSources)"
        HintNamespace="$([MSBuild]::ValueOrDefault('$(RootNamespace).%(RelativeDir)', '').Replace('\', '.').Replace('/', '.').Trim('.'))"/>

  </ItemGroup>

  <ItemGroup Label="Expose MSBuild properties to analyzer" Condition="'$(IsRazorSdkComponent)' != 'true'">
    <AvailableItemName Include="RazorSdkSources"/>
    <CompilerVisibleItemMetadata Include="AdditionalFiles" MetadataName="HintNamespace"/>
  </ItemGroup>

</Project>
```

06

Использование RazorSdk

Подключение в проект

Использование RazorSdk

- Оба пакета содержат одинаковый код
- ronimizy.Razor.Sdk – подключается как обычная библиотека
- ronimizy.Razor.Sdk.Analyzers – подключается как анализаторы Roslyn

```
<PropertyGroup>
  <IsRazorSdkComponent>true</IsRazorSdkComponent>
</PropertyGroup>

<ItemGroup>
  <PackageReference Include="ronimizy.Razor.Sdk" PrivateAssets="all"/>
  <PackageReference Include="ronimizy.Razor.Sdk.Analyzers"/>
</ItemGroup>
```

RazorSourceGeneratorBase

Использование RazorSdk

```
public abstract class RazorSourceGeneratorBase : IIncrementalGenerator
{
   public void Initialize(IncrementalGeneratorInitializationContext context);

   protected virtual void InitializeCore(IncrementalGeneratorInitializationContext context);

   protected virtual SyntaxNode ModifyCSharpSyntax(SyntaxNode csharpNode);

   protected virtual string CreateFileName(RazorSdkProjectItem projectItem);

   private static bool IsApplicableProjectItem(RazorSdkProjectItem file);

   protected virtual bool IsApplicableRazorFile(AdditionalText text);

   protected virtual void ConfigureFileSystem(VirtualRazorProjectFileSystem fileSystem);

   protected virtual void ConfigureEngineBuilder(RazorProjectEngineBuilder builder);

   protected virtual bool OnCodeDocumentCreated(
      RazorSdkProjectItem projectItem,
      RazorCodeDocument codeDocument,
      SyntaxNode syntaxNode,
      SourceProductionContext context);

   protected abstract IRazorDocumentClassifierPass CreateClassifierPass(RazorSdkProjectItem projectItem);
}
```

RazorSourceGeneratorBase

Использование RazorSdk

```
public abstract class RazorSourceGeneratorBase : IIncrementalGenerator
{
   public void Initialize(IncrementalGeneratorInitializationContext context);

   protected virtual void InitializeCore(IncrementalGeneratorInitializationContext context);

   protected virtual SyntaxNode ModifyCSharpSyntax(SyntaxNode csharpNode);

   protected virtual string CreateFileName(RazorSdkProjectItem projectItem);

   private static bool IsApplicableProjectItem(RazorSdkProjectItem file);

   protected virtual bool IsApplicableRazorFile(AdditionalText text);

   protected virtual void ConfigureFileSystem(VirtualRazorProjectFileSystem fileSystem);

   protected virtual void ConfigureEngineBuilder(RazorProjectEngineBuilder builder);

   protected virtual bool OnCodeDocumentCreated(
      RazorSdkProjectItem projectItem,
      RazorCodeDocument codeDocument,
      SyntaxNode syntaxNode,
      SourceProductionContext context);

   protected abstract IRazorDocumentClassifierPass CreateClassifierPass(RazorSdkProjectItem projectItem);
}
```

RazorSourceGeneratorBase

Использование RazorSdk

```
[Generator]
public class RazorStringBuilderGenerator : RazorSourceGeneratorBase
{
    protected override IRazorDocumentClassifierPass CreateClassifierPass(RazorSdkProjectItem projectItem)
        => new CustomDocumentClassifierPass(projectItem);
}
```

MSBuild

Использование RazorSdk

- В SDK определяются props и targets, подключаемые в транзитивные проекты
- Они подключают все .razor файлы как AdditionalFiles
- Расчтывают для всех .razor файлов HintNamespace

07

Что можно сделать кастомными компиляторами Razor?

PDF

Что можно сделать кастомными компиляторами Razor?

QuestPDF

Что можно сделать кастомными компиляторами Razor?

```
using QuestPDF.Fluent;
using QuestPDF.Helpers;
using QuestPDF.Infrastructure;

public class MyDocument : IDocument
{
    public DocumentMetadata GetMetadata() => DocumentMetadata.Default;

    public void Compose(IDocumentContainer container)
    {
        container.Page(page =>
        {
            page.Margin(40);
            page.Size(PageSizes.A4.Landscape());
            page.PageColor(Colors.White);
            page.DefaultTextStyle(x => x.FontSize(14));

            page.Header()
                .Text("My First PDF Document")
                .SemiBold().FontSize(20).FontColor(Colors.Blue.Medium);

            page.Content()
                .PaddingVertical(10)
                .Column(col =>
                {
                    col.Spacing(10);

                    col.Item().Text("Hello, this is a sample PDF generated using QuestPDF.");
                    col.Item().Text("You can layout text, images, tables, and much more.");
                    col.Item().Image(Placeholders.Image(200, 100)); // Placeholder image
                });

            page.Footer()
                .AlignCenter()
                .Text(x =>
                {
                    x.Span("Page ");
                    x.CurrentPageNumber();
                    x.Span(" of ");
                    x.TotalPages();
                });
        });
    }
}
```

~~QuestPDF.Razor~~

Что можно сделать кастомными компиляторами Razor?

```
@using QuestPDF.Fluent;
@using QuestPDF.Helpers;
@using QuestPDF.Infrastructure;
@implements IDocument;

<Document>
 <Page Size="PageSizes.A4.Landscape()" Margin="40" PageColor="Colors.White">
  <DefaultTextStyle FontSize="14"/>

  <Header>
   <Text FontSize="20" FontColor="Colors.Blue.Medium" FontWeight="FontWeight.SemiBold">
    My First PDF Document
   </Text>
  </Header>

  <Content PaddingVertical="10">
   <Column Spacing="10">
    <Text>Hello, this is a sample PDF generated using QuestPDF.</Text>
    <Text>You can layout text, images, tables, and much more.</Text>
    <Image Source="@Placeholders.Image(200, 100)"/>
   </Column>
  </Content>

  <Footer>
   <AlignCenter>
    <Text><Span>Page </Span>
     <CurrentPageNumber/>
     <Span> of </Span>
     <TotalPages/>
    </Text>
   </AlignCenter>
  </Footer>
 </Page>
</Document>
```

Таблицы

Что можно сделать кастомными компиляторами Razor?

Sales Report Q3 2025			
Product	**Region**	**Units Sold**	**Revenue**
Widget A	EMEA	1200	36000
Widget B	NA	900	27000 (after discount)
Widget C	APAC	1600	48000

```
public record ProductData(
    string Product,
    string Region,
    int UnitsSold,
    decimal Revenue,
    string? RevenueComment);
```

FluentSpreadsheets

Что можно сделать кастомными компиляторами Razor?

```
public class CustomTable : RowTable<IReadOnlyCollection<ProductData>>
{
    protected override IEnumerable<IRowComponent> RenderRows(
        IReadOnlyCollection<ProductData> model)
    {
        yield return Row(
            Label("Sales Report Q3 2025")
                .WithTextKind(TextKind.Bold)
                .WithBottomBorder(BorderType.Medium, Color.DarkGray)
        );

        yield return Row(
            Label("Product").WithTextKind(TextKind.Bold).WithColumnWidth(2),
            Label("Region").WithTextKind(TextKind.Bold),
            Label("Units Sold").WithTextKind(TextKind.Bold),
            Label("Revenue").WithTextKind(TextKind.Bold)
        );

        foreach (ProductData data in model)
        {
            var revenue = string.IsNullOrEmpty(data.RevenueComment)
                ? Label(data.Revenue)
                : VStack(Label(data.Revenue), Label(data.RevenueComment));

            revenue = revenue.WithContentAlignment(HorizontalAlignment.Trailing, VerticalAlignment.Center);

            yield return Row(
                Label(data.Product),
                Label(data.Region).WithContentAlignment(HorizontalAlignment.Center, VerticalAlignment.Center),
                Label(data.UnitsSold).WithContentAlignment(HorizontalAlignment.Trailing, VerticalAlignment.Center),
                revenue
            );
        }
    }

    protected override IComponent Customize(IComponent component) => component.WithAdjustedColumns();
}
```

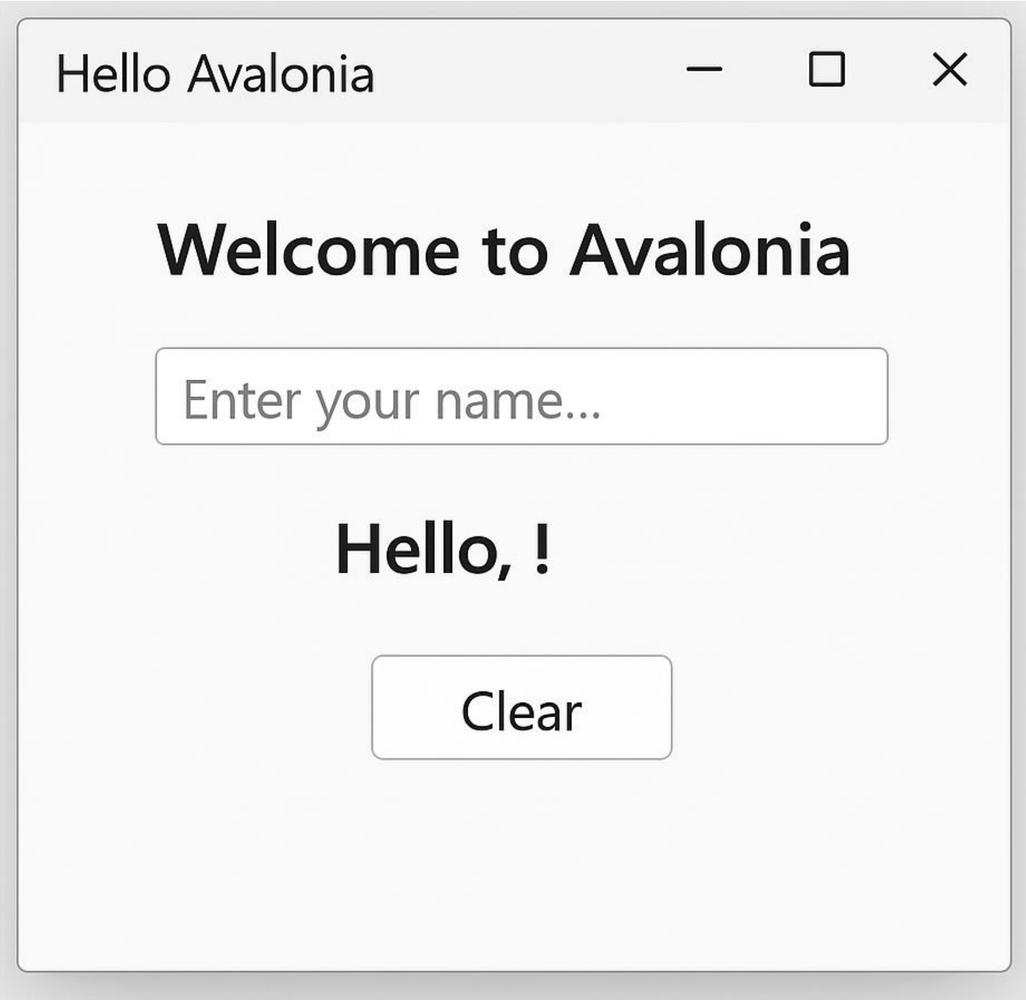
~~FluentSpreadsheets.Razor~~

Что можно сделать кастомными компиляторами Razor?

```
@using FluentSpreadsheets
@inherits RowTable<IReadOnlyCollection<ProductData>>

<Row>
  <Label TextKind="Bold" BottomBorder="Medium" BorderColor="DarkGray">
    Sales Report Q3 2025
  </Label>
</Row>

<Row>
  <Label TextKind="Bold" ColumnWidth="2">Product</Label>
  <Label TextKind="Bold">Region</Label>
  <Label TextKind="Bold">Units Sold</Label>
  <Label TextKind="Bold">Revenue</Label>
</Row>

@foreach (var data in model)
{
  <Row>
    <Label>@data.Product</Label>
    <Label HorizontalAlign="Center" VerticalAlign="Center">@data.Region</Label>
    <Label HorizontalAlign="Trailing" VerticalAlign="Center">@data.UnitsSold</Label>

    @if (!string.IsNullOrEmpty(data.RevenueComment))
    {
      <VStack HorizontalAlign="Trailing" VerticalAlign="Center">
        <Label>@data.Revenue</Label>
        <Label TextKind="Italic" TextColor="Gray">@data.RevenueComment</Label>
      </VStack>
    }
    else
    {
      <Label HorizontalAlign="Trailing" VerticalAlign="Center">@data.Revenue</Label>
    }
  </Row>
}

@code {
  protected override IComponent Customize(IComponent component) => component.WithAdjustedColumns();
}
```

Avalonia

Что можно сделать кастомными компиляторами Razor?



Avalonia

Что можно сделать кастомными компиляторами Razor?

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d"
        x:Class="MyApp.Views.MainWindow"
        Title="Hello Avalonia" Width="420" Height="220">
    <StackPanel Margin="20" Spacing="12">
        <TextBlock Text="Welcome to Avalonia" FontSize="20" />
        <TextBox Watermark="Enter your name…" Text="{Binding Name, Mode=TwoWay}" />
        <TextBlock>
            <Run Text="Hello, " />
            <Run Text="{Binding Name}" />
            <Run Text="!" />
        </TextBlock>
        <Button Content="Clear" Command="{Binding ClearCommand}" />
    </StackPanel>
</Window>
```

~~Avalonia.Razor~~

Что можно сделать кастомными компиляторами Razor?

```
@using System
@using Avalonia.Controls
@using Avalonia.Layout
@using Avalonia.Media
@using MyApp.ViewModels
@inherits Window<MainViewModel>

<StackPanel Margin="20" Spacing="12">
  <TextBlock Text="Welcome to Avalonia" FontSize="20"/>
  <TextBox Watermark="Enter your name…" @bind-Text="@Model.Name"/>
  <TextBlock>
    <Run Text="Hello, "/>
    <Run Text="@Model.Name"/>
    <Run Text="!"/>
  </TextBlock>
  <Button Content="Clear" Command="@Model.ClearCommand"/>
</StackPanel>

@code {
  public override string Title => "Hello Avalonia";
  public override double Width => 420;
  public override double Height => 220;
}
```

~~Avalonia.Razor~~

Что можно сделать кастомными компиляторами Razor?

xaml/axaml

```
<TextBox Watermark="Enter your name…" Text="{Binding Name, Mode=TwoWay}" />
```

razor

```
<TextBox Watermark="Enter your name…" @bind-Text=“@Model.Name"/>
```

ronimizy.RazorStringBuilder

Что можно сделать кастомными компиляторами Razor?

```
@inherits RazorStringBuilder

update table
set updated_at = now()
@if (string.IsNullOrEmpty(Data) is false)
{
    @:, data = '@Data'
}
where id = :id

@code {
    public required string? Data { get; set; }
}
```

```
1
5   var query = new UpdateQuery { Data = "aa" };
1
2   var builder = new StringBuilder();
3   query.Build(builder);
4
5   Console.Clear();
6   Console.WriteLine(builder.ToString());
7
```

Run RazorStringBuilder.Sample

```
update table
set updated_at = now()
    , data = 'aa'
where id = :id
```

08 Выводы

Выводы

- Исходный код компилятора Razor в результате проделанной работы можно переделать для компиляции Razor под собственный таргет

Выводы

- Исходный код компилятора Razor в результате проделанной работы можно переделать для компиляции Razor под собственный таргет
- С точки зрения шаблонизации строк, есть ограничения с точки зрения синтаксического дерева Razor

Выводы

- Исходный код компилятора Razor в результате проделанной работы можно переделать для компиляции Razor под собственный таргет
- С точки зрения шаблонизации строк, есть ограничения с точки зрения синтаксического дерева Razor
- Тулинг IDE (в частности Rider) не поддерживает Razor как first-class citizen

Что дальше?

Выводы

- Написать документацию к Razor.Sdk
- Реализовать компилятор из Razor во FluentSpreadsheets

Ссылки

Выводы

- https://github.com/dotnet/razor
- https://github.com/ronimizy/razor-sdk
- https://github.com/ronimizy/razor-string-builder

- https://github.com/QuestPDF/QuestPDF
- https://github.com/ronimizy/FluentSpreadsheets
- https://github.com/AvaloniaUI/Avalonia